

Aufgabe 1.1

In Aufgabe 1.1 wurde der `AntlrTokenCollector` implementiert, der mithilfe des ANTLR-Lexers alle Tokens eines MiniJava-Programms einliest und passende Highlight-Regionen erzeugt. Zunächst wird der Eingabetext in einen `CharStream` umgewandelt und an den `MiniJavaLexer` übergeben. Anschließend holen wir alle Tokens und iterieren über sie. Kommentare werden über den `HIDDEN_CHANNEL` erkannt und je nach Typ (Line-, Block- oder Javadoc-Kommentar) mit der entsprechenden Farbe markiert. Danach prüfen wir, ob ein Token ein Keyword ist und erzeugen dafür eine Highlight-Region. Auch String- und Character-Literale werden farblich hervorgehoben. Annotationen werden durch das '@'-Token erkannt; zusätzlich wird geprüft, ob danach ein Identifier folgt, der ebenfalls markiert wird. Identifier selbst werden nicht hervorgehoben, da keine Farbe dafür definiert ist. Am Ende geben wir die gesammelten Highlight-Regionen zurück.

Aufgabe 1.2

1. Methoden implementieren

1.1. `visitCompilationUnit`

- package-Deklaration (falls vorhanden) → eine Zeile
- import-Deklarationen → jeweils eine Zeile
- zwischen package/import/type → **Leerzeilen**
- type-Deklarationen → nacheinander ausgeben

1.2. `visitClassBody`

- { ausgeben → `nl()`
- `currentIndent++`
- alle `classBodyDeclaration`-Kinder besuchen → jeweils eigene Zeile
- `currentIndent--`
- } ausgeben → `nl()`

1.3. `visitBlock`

- exakt wie `classBody`, aber für Blöcke in Methoden/if/while

1.4. visitStatement

- Wenn Statement ein Block ist → visitBlock
- Wenn Statement `return ... ;` → eine Zeile
- Wenn Statement `expression ;` → eine Zeile
- Wenn Statement `if (...) statement` → Struktur beachten:
 - `if (...)` → Tokens ausgeben
 - Statement besuchen
 - optional `else` → Tokens ausgeben + Statement besuchen

Aufgabe 1.3

In Aufgabe 1.3 wurde eine kleine Anwendung erstellt, die den zuvor implementierten `PrettyPrinterVisitor` auf Beispielprogramme anwendet. Ziel war es, den vollständigen Ablauf vom Einlesen eines MiniJava-Programms über das Parsen bis hin zur formatierten Ausgabe zu demonstrieren. Dazu wird zunächst ein `CharStream` aus einer Eingabedatei oder einem String erzeugt und an den ANTLR-Lexer übergeben. Der Lexer erzeugt Tokens, die anschließend vom Parser verarbeitet werden, um einen vollständigen Parse-Tree zu erzeugen. Dieser Parse-Tree wird anschließend vom `PrettyPrinterVisitor` besucht, der den strukturierten, korrekt eingerückten und formatierten MiniJava-Code erzeugt. Die Anwendung gibt das Ergebnis schließlich auf der Konsole aus.